Processor Review

CS 2410

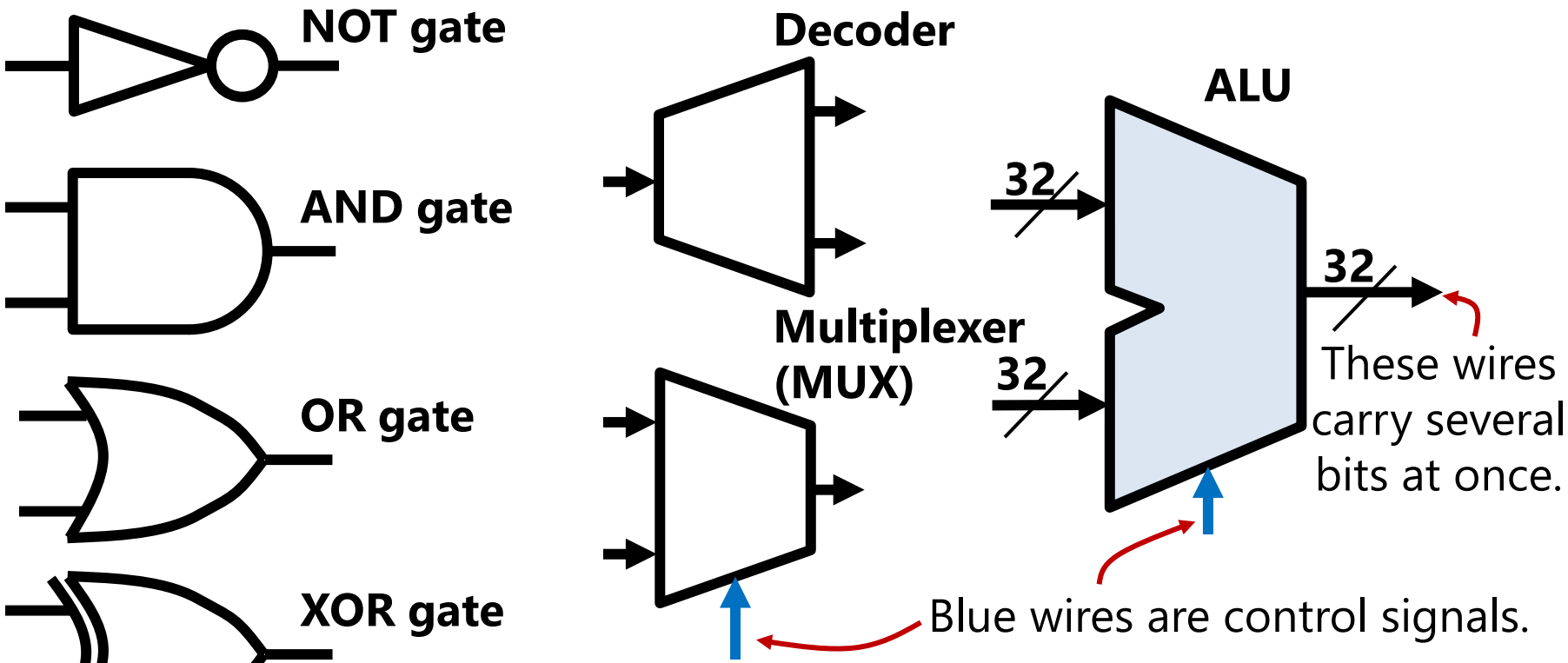
Wonsun Ahn

University of Pittsburgh

Clocking Review

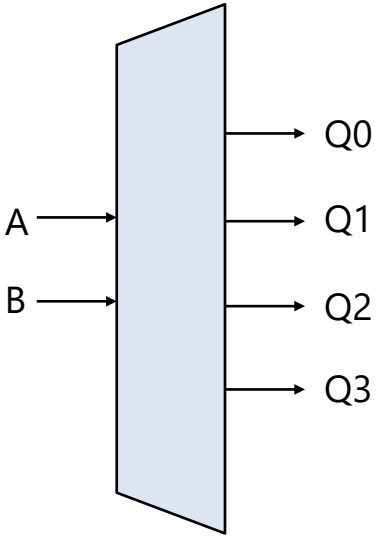
Logic components

- Do you remember what all these do?



Uses of a Decoder

- Translates a set of input signals to a bunch of output signals.
  - Register decoder: selects one register given register number



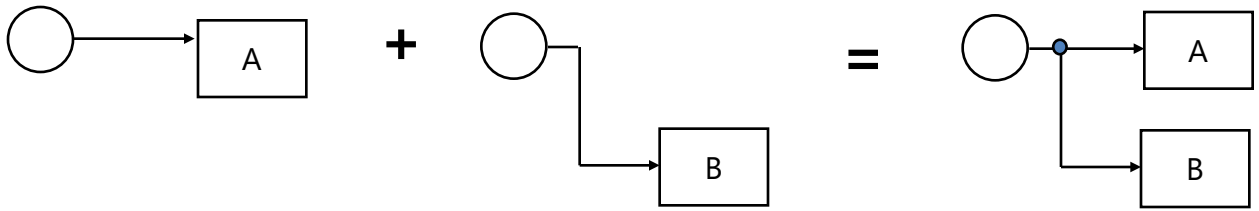
Truth Table for Decoder

A	B	Q0	Q1	Q2	Q3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

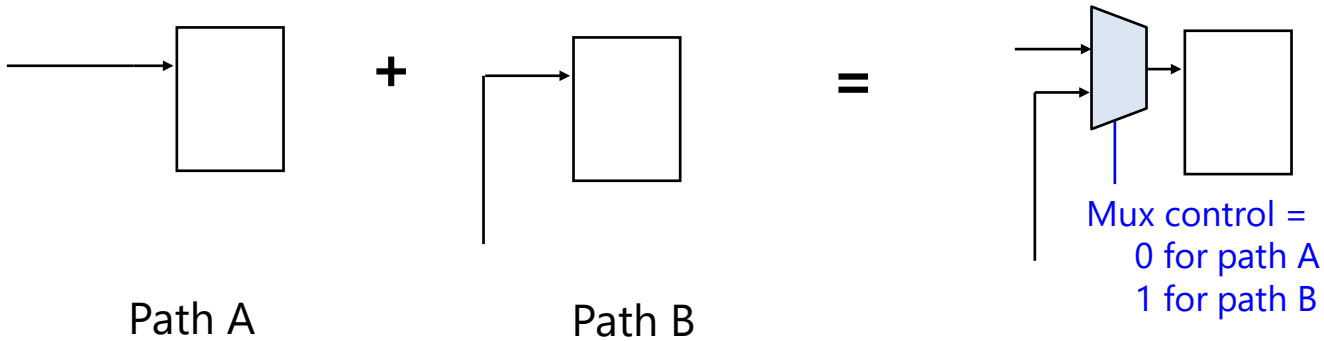
  - Instruction decoder: generates control signals from instructions
  - Any truth table can be translated to an optimal set of gates
    - For the above: $Q0 = \overline{AB}, Q1 = \bar{A}B, Q2 = A\bar{B}, Q3 = AB$

Uses of a Multiplexer

- No problem in fanning out one signal to two points

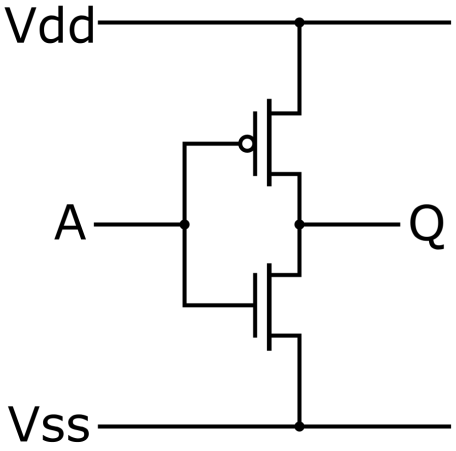


- Cannot connect two signals to one point
  - Must use a multiplexer to *select* between the two

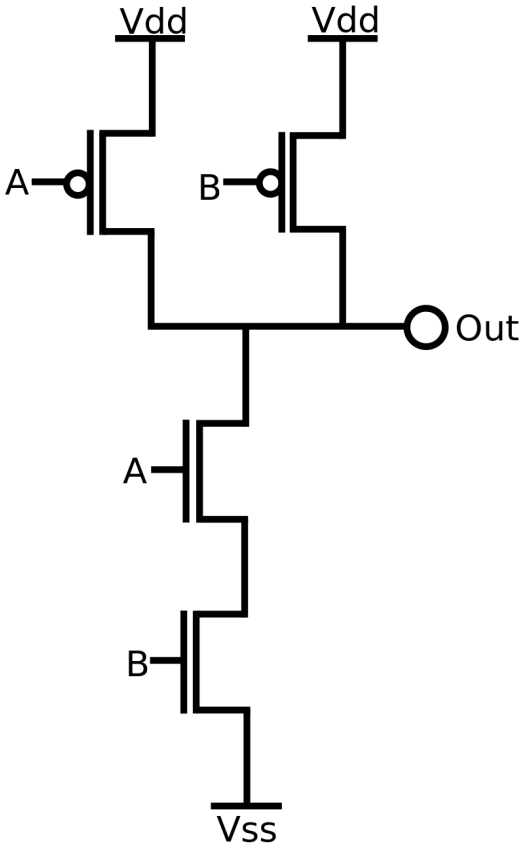


Gates are made of transistors (as noted previously)

- NOT gate

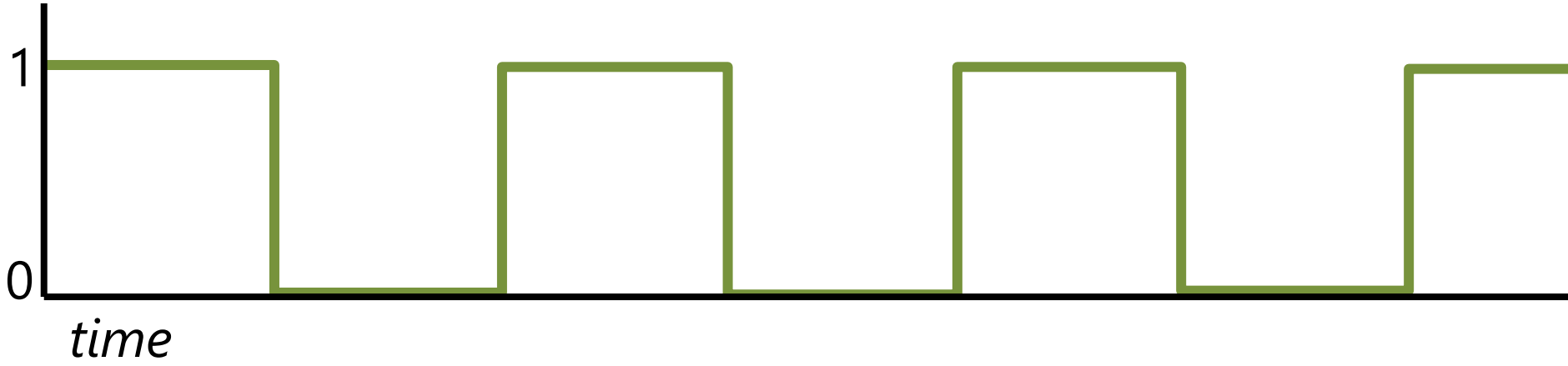


- NAND gate



The clock signal

- The clock is a signal that alternates regularly between 0 and 1:

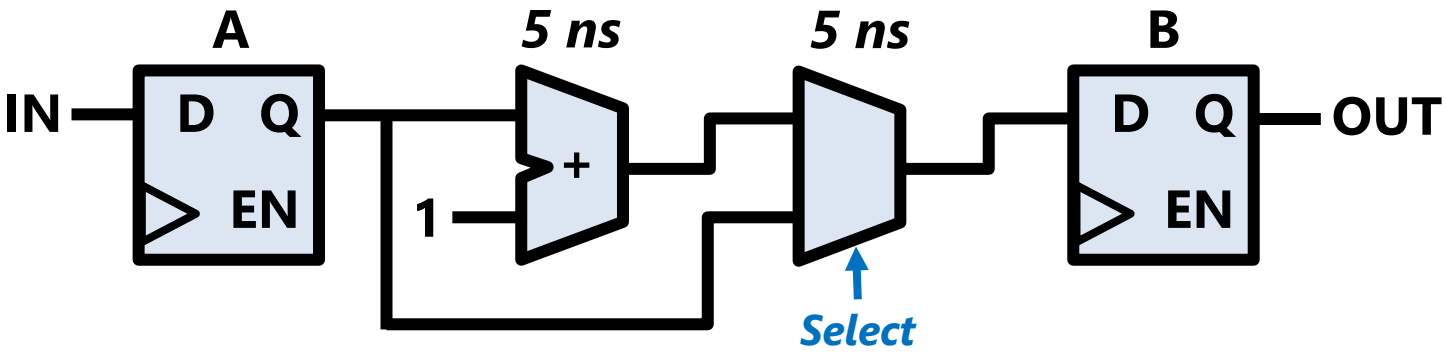


- Bits are latched on to registers and flip-flops on rising edges
- In between rising edges, bits propagate through the logic circuit

Critical Path

- ***Propagation Delay***: delay to propagate data to next storage location
- ***Critical Path***: path in a circuit that has longest propagation delay
  - $PropagationDelay_{CriticalPath} < CycleTime$

- How fast can we clock this circuit?



  - Is it 1 / 5 ns (5×10^{-9}s) = 200 MHz?
  - Or is it 1 / 10 ns (10×10^{-9}s) = 100 MHz? ✓

MIPS Review

RISC vsa CISC

	RISC (Reduced Instruction Set Computer)	CISC (Complex Instruction Set Computer)
Design Philosophy	Many simple generic instructions to encode the same amount of work	A few specialized instructions to encode the same amount of work
Example Instructions	load, store, add, mult	sin, cos, tan, linear algebra (in addition to load, store, add, mult)
Instruction Length	Fixed (32-bit or 64-bit)	Variable (to accommodate complex instructions with variable operands)
Code Density	**Low** (high code memory usage)	**High** (low code memory usage)
Specialization Opportunity	**Low** (must translate sin into loops using adds and mults)	**High** (can create a specialized integrated circuit for calculating sin)
Instruction Parallelization	**Easy** to fetch many fixed length insts **Easy** to parallelize regular instructions	**Hard** to fetch many variable length insts **Hard** to parallelize complex instructions
Amenable to Compilers	**Easy** to generate simple instructions **Easy** to optimize simple instructions	**Hard** to map algorithms in high-level languages to complex instructions

RISC vs. CISC: Who Won?

- For general purpose computing: RISC
  - RISC architectures: MIPS, SPARC, PowerPC, ARM, RISC-V
  - CISC architecture: x86 (for legacy code reasons only)
    - x86 CPUs decode complex insts into uops before execution

- Why did this happen (circa 1980)?
  - **Code density**: with Moore's law, code memory size no longer a concern
  - **Specialization**: with usage of high-level languages and compilers, usage of specialized instructions through assembly programming plummeted
    - Recall: *Use abstraction to simplify design*
  - **Instruction parallelization**: gains through instruction parallelization in RISC beat gains through instruction specialization in CISC
    - Recall: *Performance via parallelization*
  - **Compilers**: Engineering effort to use specialized instructions could be better spent in optimizing generic instructions common to all ISAs
    - Recall: *Make the common case fast*

Is CISC dead? (No.)

- For specialized accelerators and extensions CISC still makes sense!

- Vector extensions added to most RISC ISAs
  - Vector add, vector mult, …
  - One vector instruction performs op on multiple vector elements
  - Occurs often enough in HPC to justify the specialization
- TPUs and NPU accelerators incorporate CISC instructions
  - Matrix multiply, activation functions, …
  - Deep learning computation is composed mostly of the above

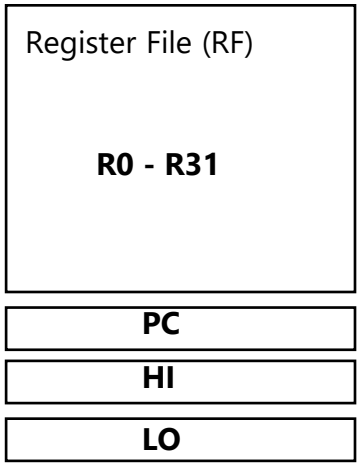
- Specialized libraries/frameworks provide abstraction for programmers
  - BLAS (Basic Linear Algebra Subroutines), OpenMP
  - Pytorch, Tensorflow

The MIPS ISA

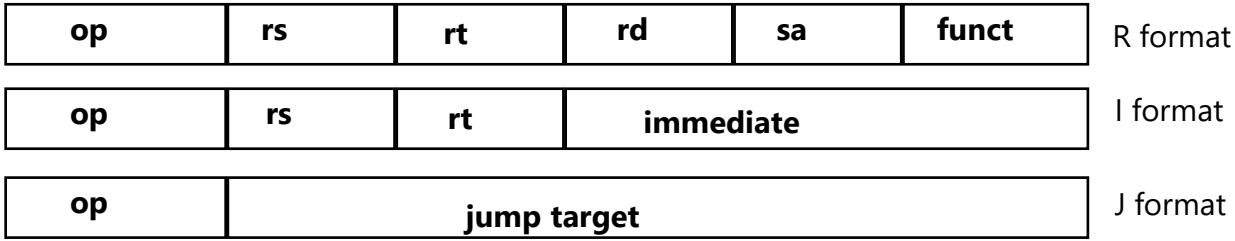
- MIPS is a ***RISC (reduced instruction set computer)*** architecture

- Instruction Categories
  - Computational
  - Load/Store
  - Jump and Branch
  - Floating Point
  - Memory Management
  - Special

Registers



3 Instruction Formats: all 32 bits wide



University of Pittsburgh

The MIPS ISA - Instruction Fields

- MIPS fields are given names to make them easier to refer to

op	rs	rt	rd	shamt	funct

op	6-bits	opcode that specifies the operation
rs	5-bits	register file address of the first source operand
rt	5-bits	register file address of the second source operand
rd	5-bits	register file address of the result's destination
shamt	5-bits	shift amount (for shift instructions)
funct	6-bits	function code augmenting the opcode

The MIPS ISA - Registers

- MIPS has 32 32-bit registers, with the following usage conventions
  - But really, all are general purpose registers (nothing special about them)

Name	Register number	Usage
`$zero`	`0`	the constant value 0 (can't be written)
`$at`	`1`	assembler temporary
`$v0-$v1`	`2-3`	values for results and expression evaluation
`$a0-$a3`	`4-7`	function arguments
`$t0-$t7`	`8-15`	unsaved temporaries
`$s0-$s7`	`16-23`	saved temporaries (like program variables)
`$t8-$t9`	`24-25`	more unsaved temporaries
`$k0-$k1`	`26-27`	reserved for OS kernel
`$gp`	`28`	global pointer
`$sp`	`29`	stack pointer
`$fp`	`30`	frame pointer
`$ra`	`31`	return address

The MIPS ISA - Memory

- MIPS is also a ***load-store*** architecture
  - **All** memory accesses performed by load and store instructions
- Memory is a giant array of 2^{32} bytes

Addr	Data
0	0x3F
1	0x00
2	0x2A
3	0x08
4	0x47
5	0xF4
6	0x26
7	0xB9
...	...

Addr	Data
0	0x3F00
2	0x2A08
4	0x47F4
6	0x26B9
...	...

Addr	Data
0	0x3F002A08
4	0x47F426B9
...	...

- The same memory viewed as bytes, 16-bit halfwords, and 32-bit words (using big-endian)
- All addresses are ***aligned*** (multiples of data size)

The MIPS ISA - Memory

- Loads move data *from* memory *into* the registers.

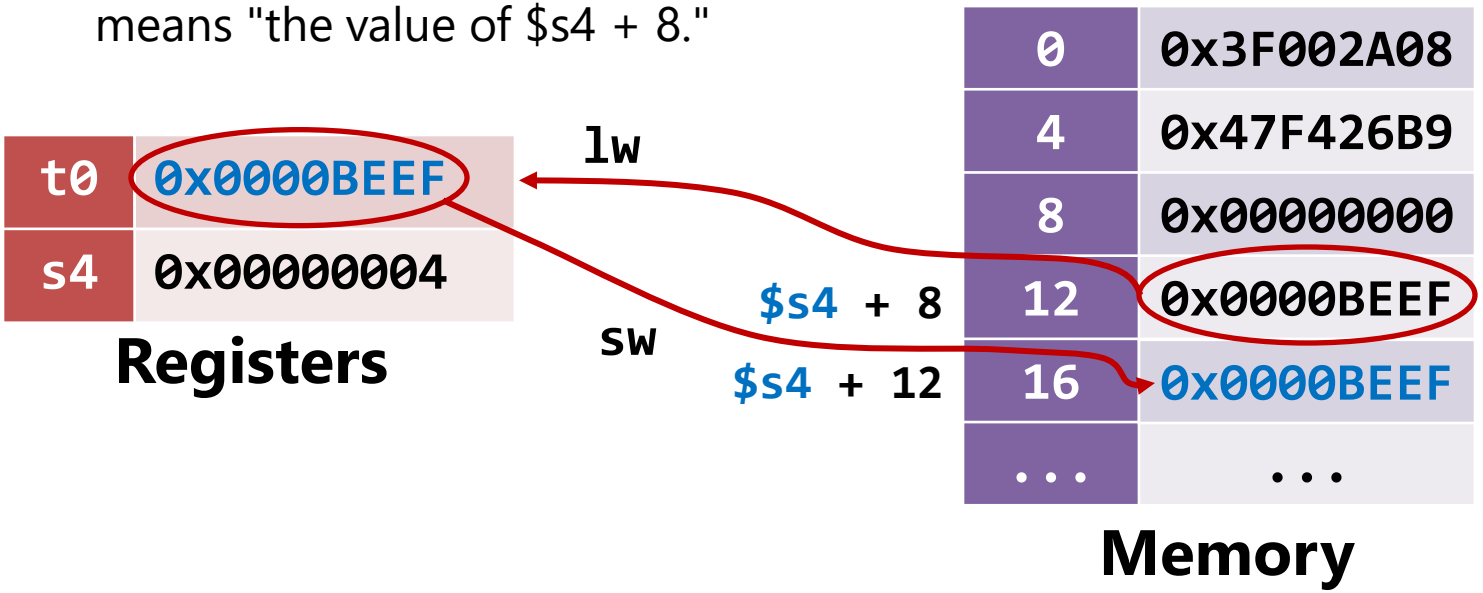
```
lw $t0, 8($s4)
```

This is the address, and it means "the value of $s4 + 8."

- Stores move data *from* the registers *into* memory.

```
sw $t0, 12($s4)
```

$t0 is the SOURCE!



The MIPS ISA – Flow control

- Jump and branch instructions change the flow of execution.

```
_top:
    # ....
    # lots o' code
    # ....
    j _top
```

```
    li   $s0, 10
_loop:
    # ....
    addi $s0, $s0, -1
    bne  $s0, $zero, _loop
    jr   $ra
```

j : jumps *unconditionally*

- jumps to **_top**

bne : jumps *conditionally*

If **$s0** != **$zero**, jumps to **_loop**

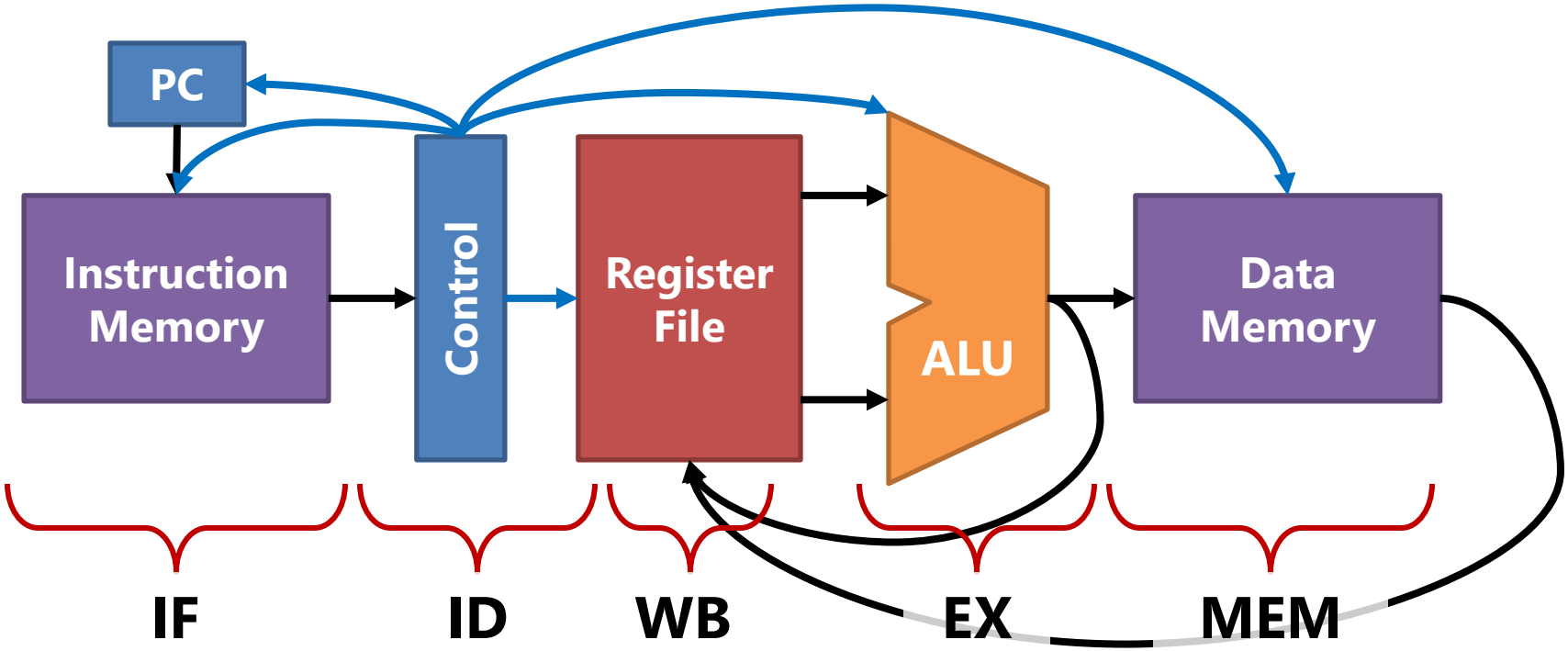
If **$s0** == **$zero**, continues to **jr $ra**

Phases of instruction execution

- In most architectures, there are five phases:
  1. **IF** (Instruction Fetch) – get next instruction from memory
  2. **ID** (Instruction Decode) – figure out what instruction it is
  3. **EX** (Execute – ALU) – do any arithmetic
  4. **MEM** (Memory) – read or write data from/to memory
  5. **WB** (Register Writeback) – write any results to the registers

- Sometimes these phases are chopped into smaller stages

A simple single-cycle implementation



- An instruction goes through IF/ID/EX/MEM/WB in one cycle

"Minimal MIPS"

It's a "subset" of MIPS

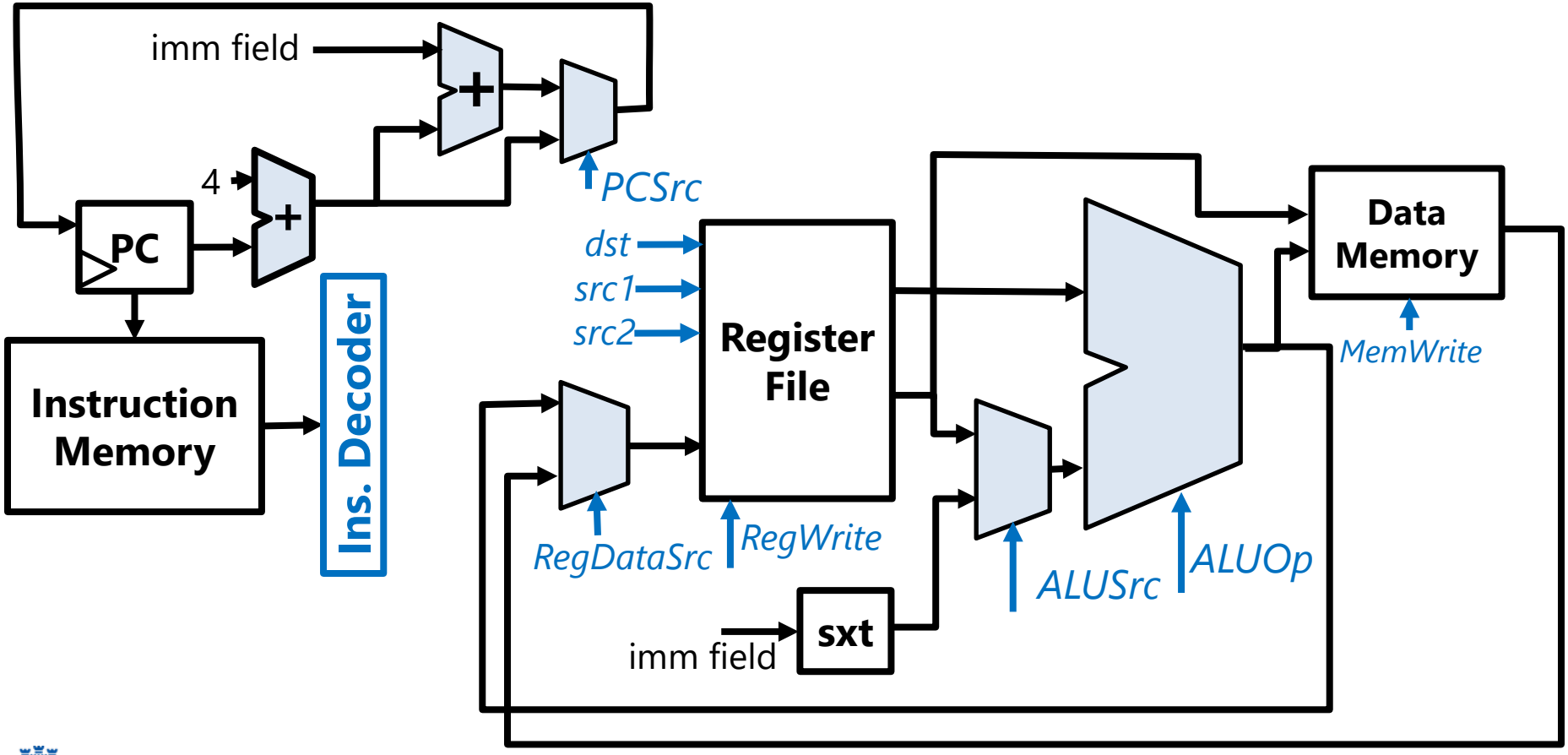
- For pedagogical (teaching) purposes

- Contains only a minimal number of instructions:
  - **lw, sw, add, sub, and, or, slt, beq,** and **j**
  - Other instructions in MIPS are variations on these anyway

- Let's review the Minimal MIPS CPU focusing on the control signals
  - Again, these control signals are decoded from the instruction

The Minimal MIPS single-cycle CPU

- A more detailed view of the 5-phase implementation



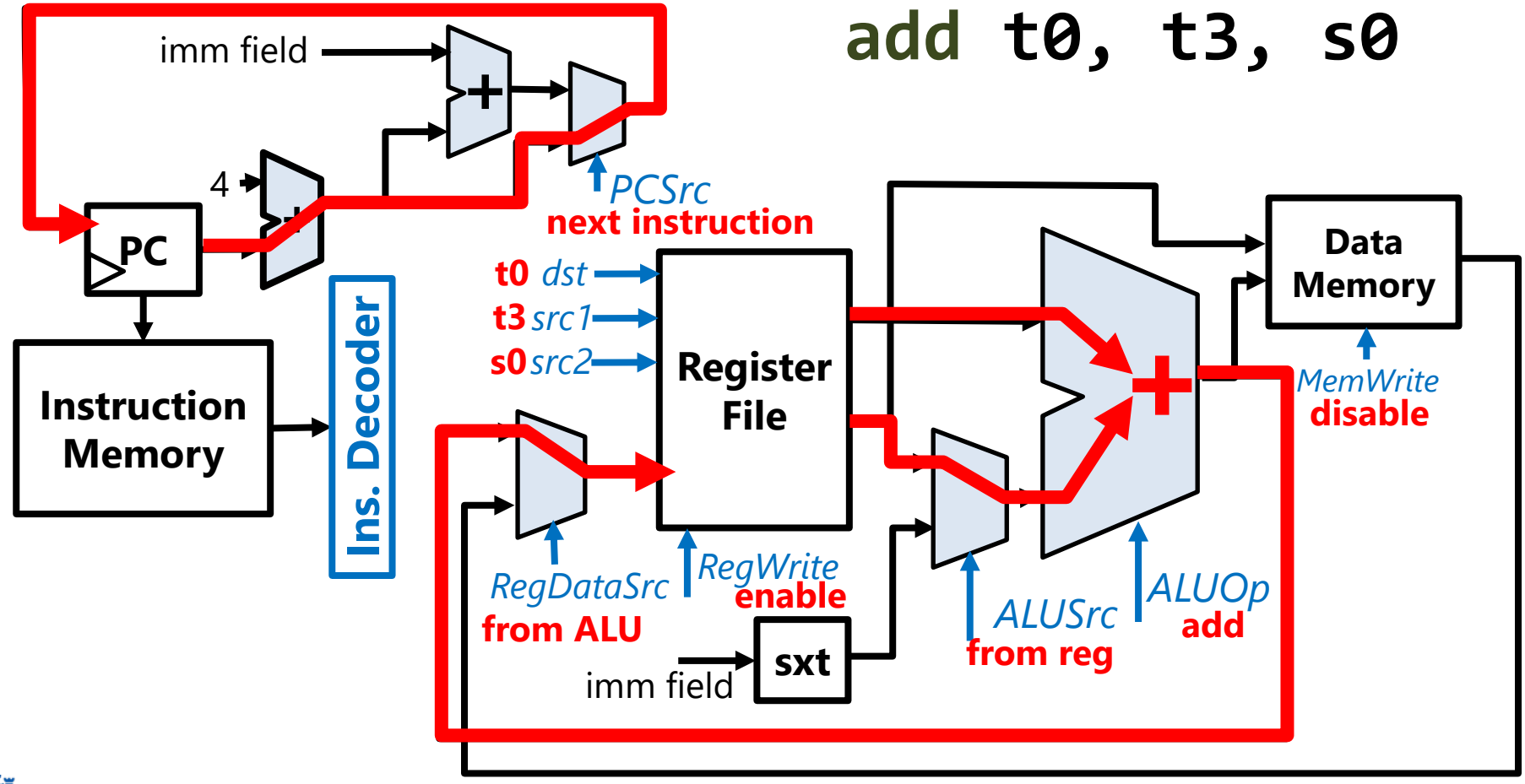
Control signals

- Registers
  - **RegDataSrc**: controls source of a register write (ALU / memory)
  - **RegWrite**: enables a write to the register file
  - **src1, src2, dst**: the register number for each respective operand
- ALU
  - **ALUSrc**: whether second operand of ALU is a register / immediate
  - **ALUOp**: controls what the ALU will do (add, sub, and, or etc)
- Memory
  - **MemWrite**: enables a write to data memory
- PC
  - **PCSrc**: controls source of next PC (PC + 4 / PC + 4 + imm)

→ All these signals are decoded from the instruction!

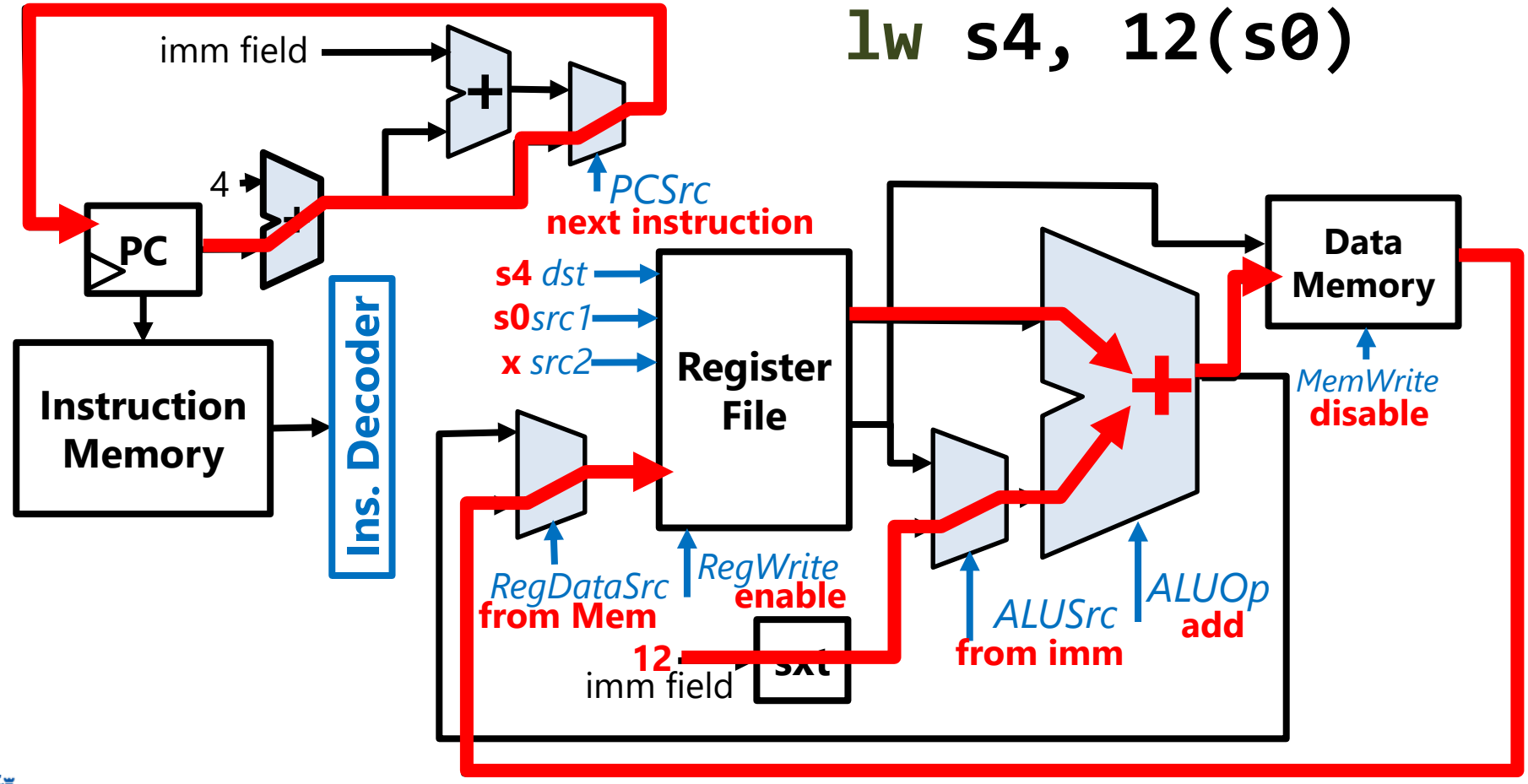
How an **add/sub/and/or/slt** work

```
add t0, t3, s0
```

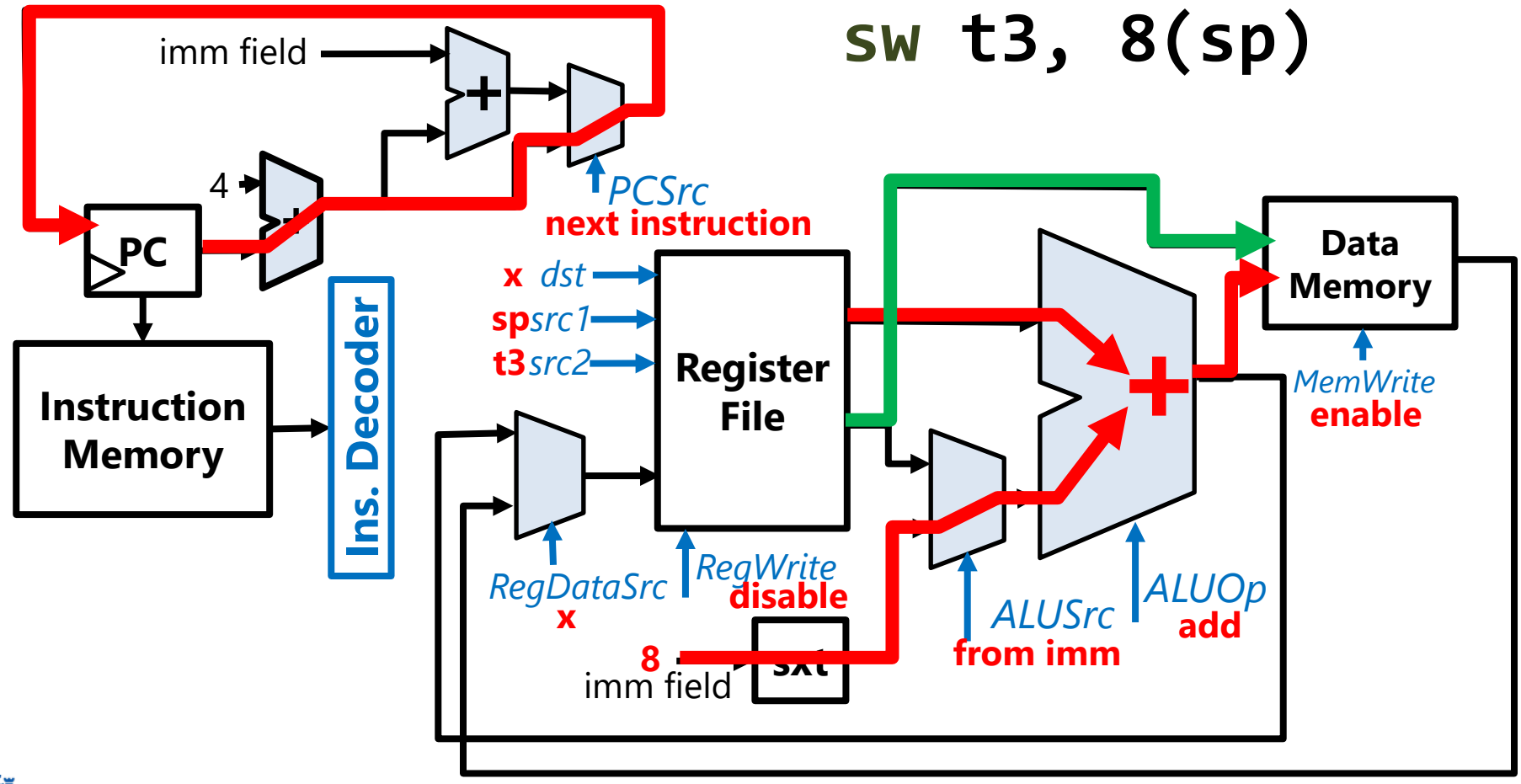


How an **lw** works

```
lw s4, 12(s0)
```

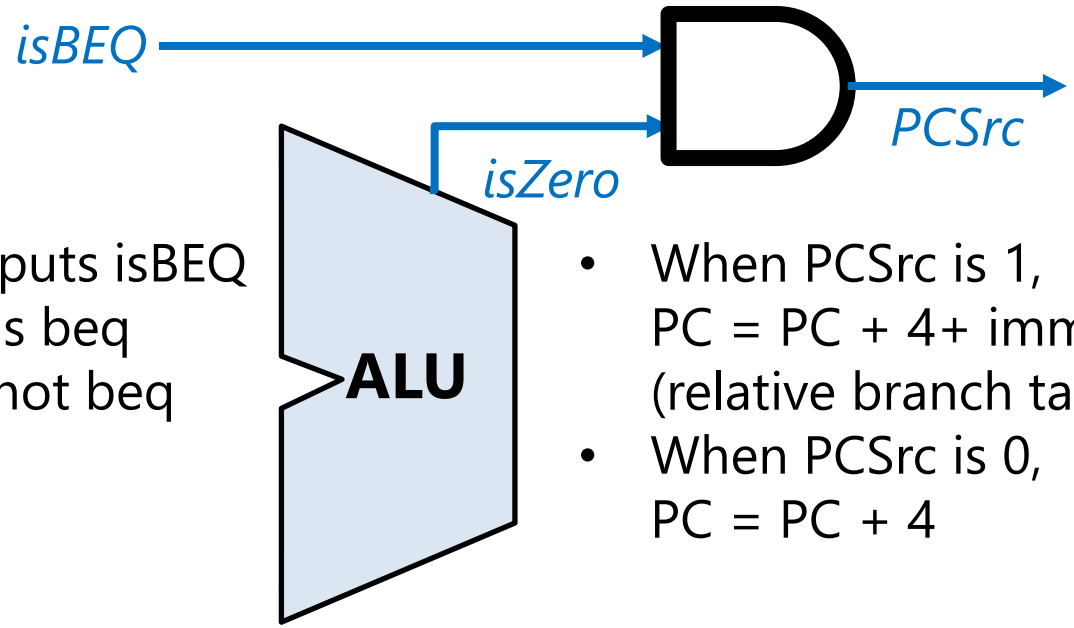


How an **sw** works



What about **beq**?

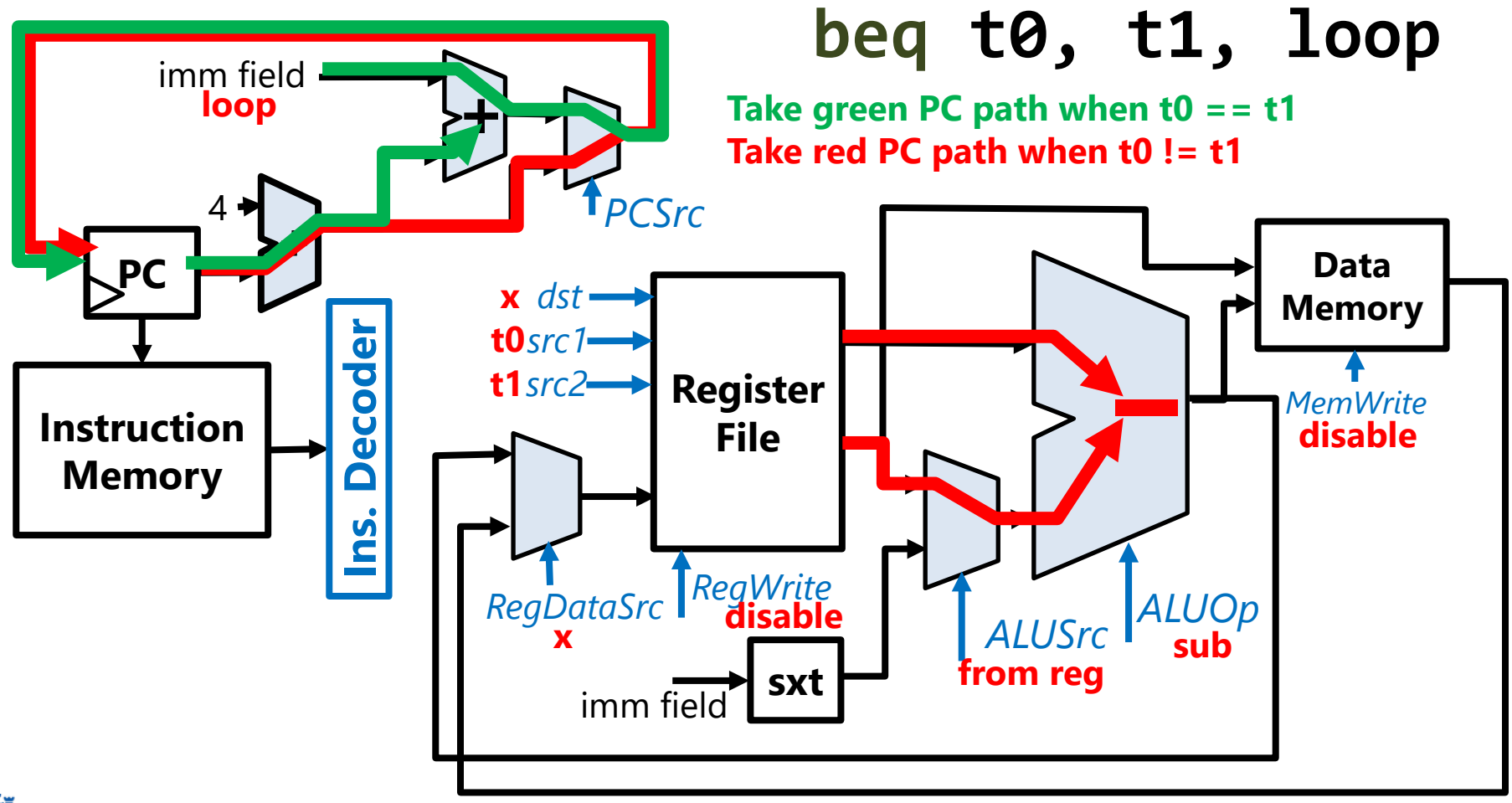
- Compares numbers by subtracting and see if result is 0
  - If result is 0, we set PCSrc to use the branch target.
  - Otherwise, we set PCSrc to PC + 4.



- Instruction decoder outputs isBEQ
  - 1: When instruction is beq
  - 0: When instruction not beq

- When PCSrc is 1, PC = PC + 4+ imm (relative branch target)
- When PCSrc is 0, PC = PC + 4

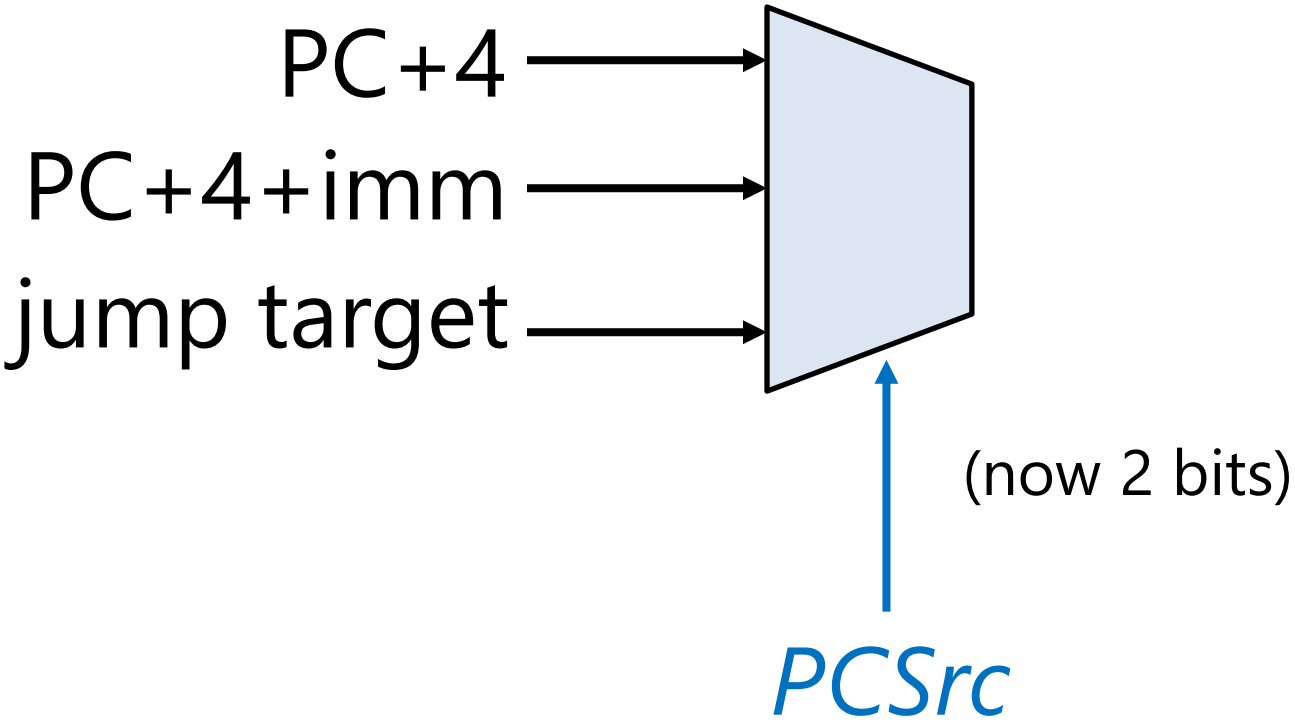
How a **beq** works



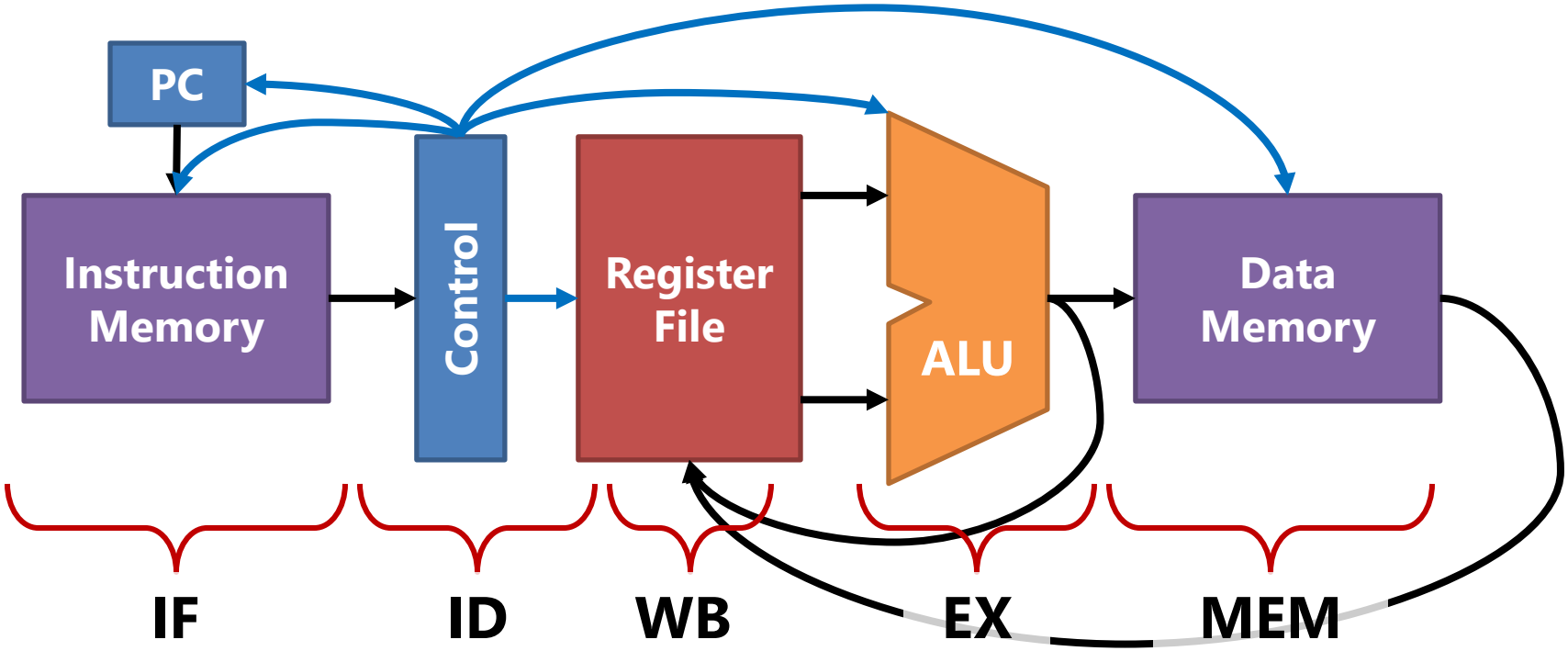
What about **j**?

- We have to add another input to the PCSrc mux.

```
j   top
```



A Single-cycle Implementation is not Optimal



- Why? Since the ***longest*** critical path must be chosen for cycle time
  - And there is a wide variation among different instructions

A Single-cycle Implementation is not Optimal

- In our CPU, the **lw** instruction has the longest critical path
  - Must go through all 5 stages: IF/ID/EX/MEM/WB
  - Whereas **add** goes through just 4 stages: IF/ID/EX/WB

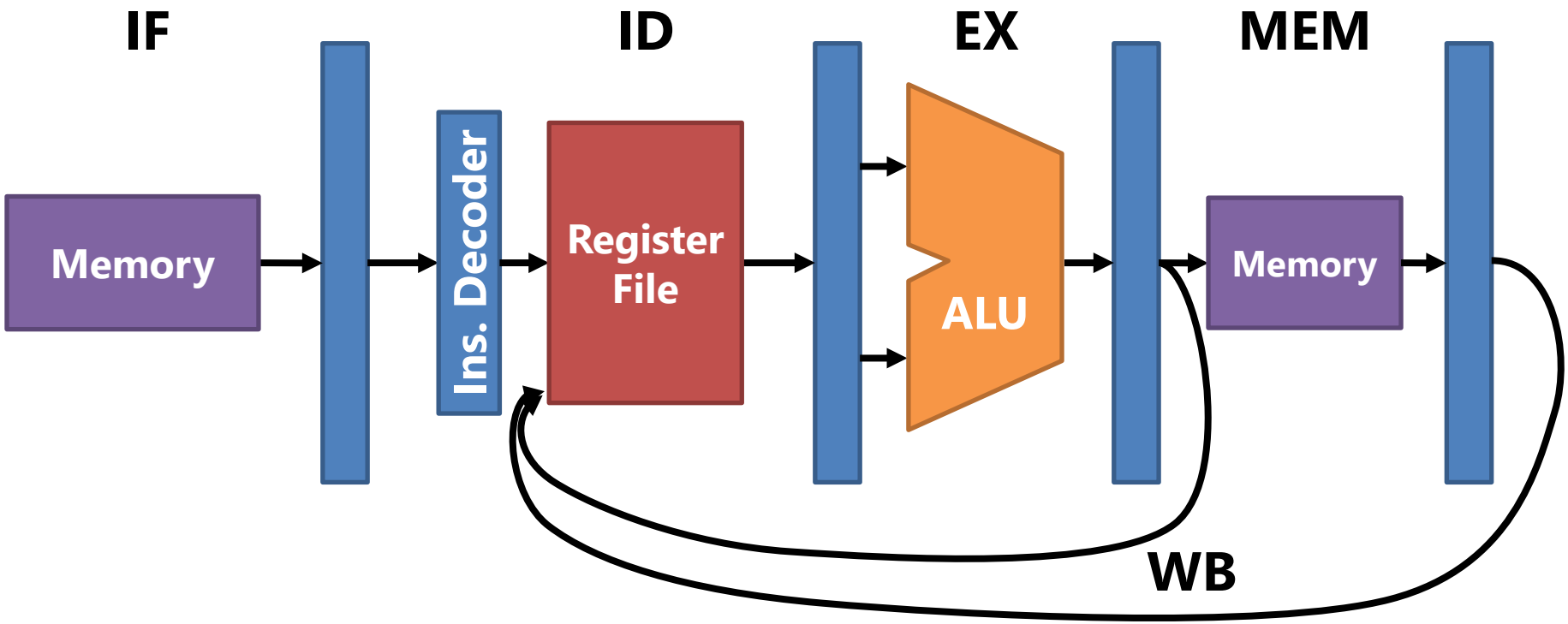
- If each phase takes *1 ns* each, cycle time must be *5 ns*:
  - Because it needs to be able to handle **lw**, which takes *5 ns*
  - **add** also takes *5 ns* when it could have been done in *4 ns*

Q) If **lw** is 1% and **add** is 99% of instruction mix,
what is the average instruction execution time?

A) Still *5 ns*! Even if **lw** is only 1% of instructions!

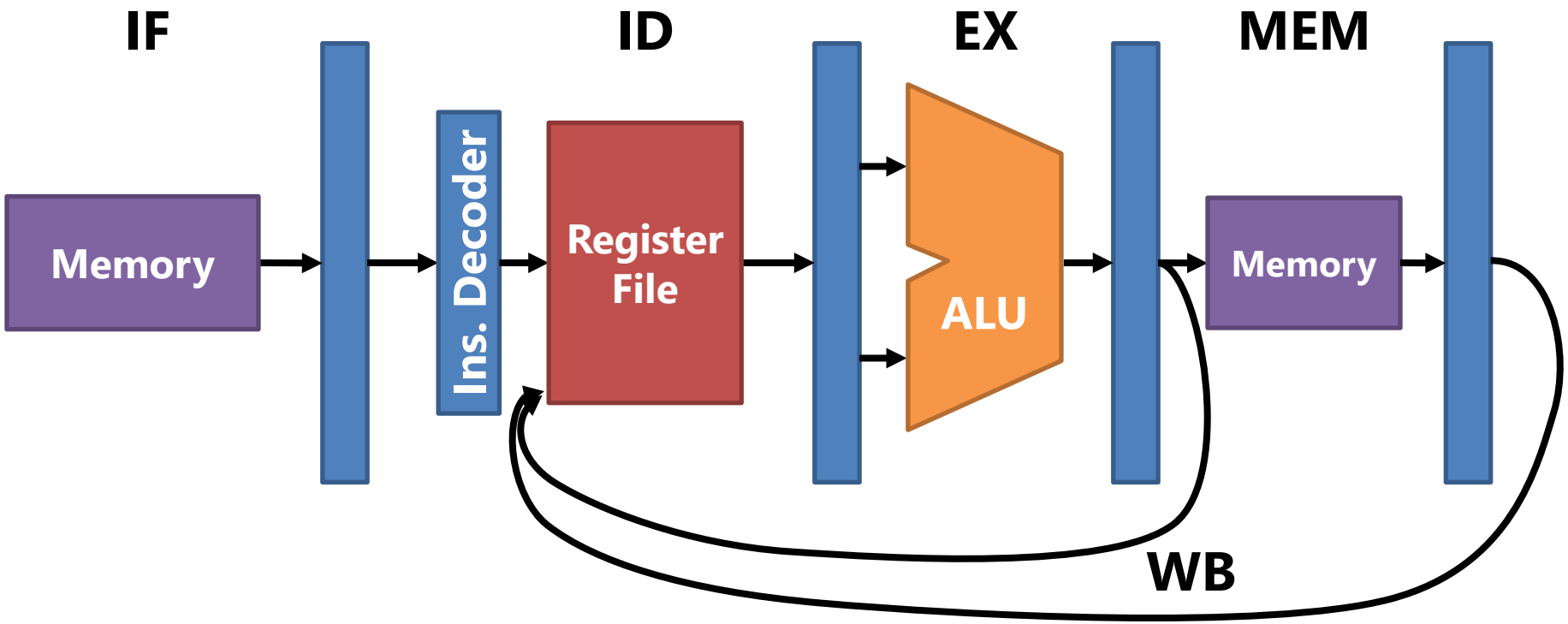
A Multi-cycle Implementation

- Internal latches store data needed for that stage
  - E.g. Latch before EX stores operands to ALU



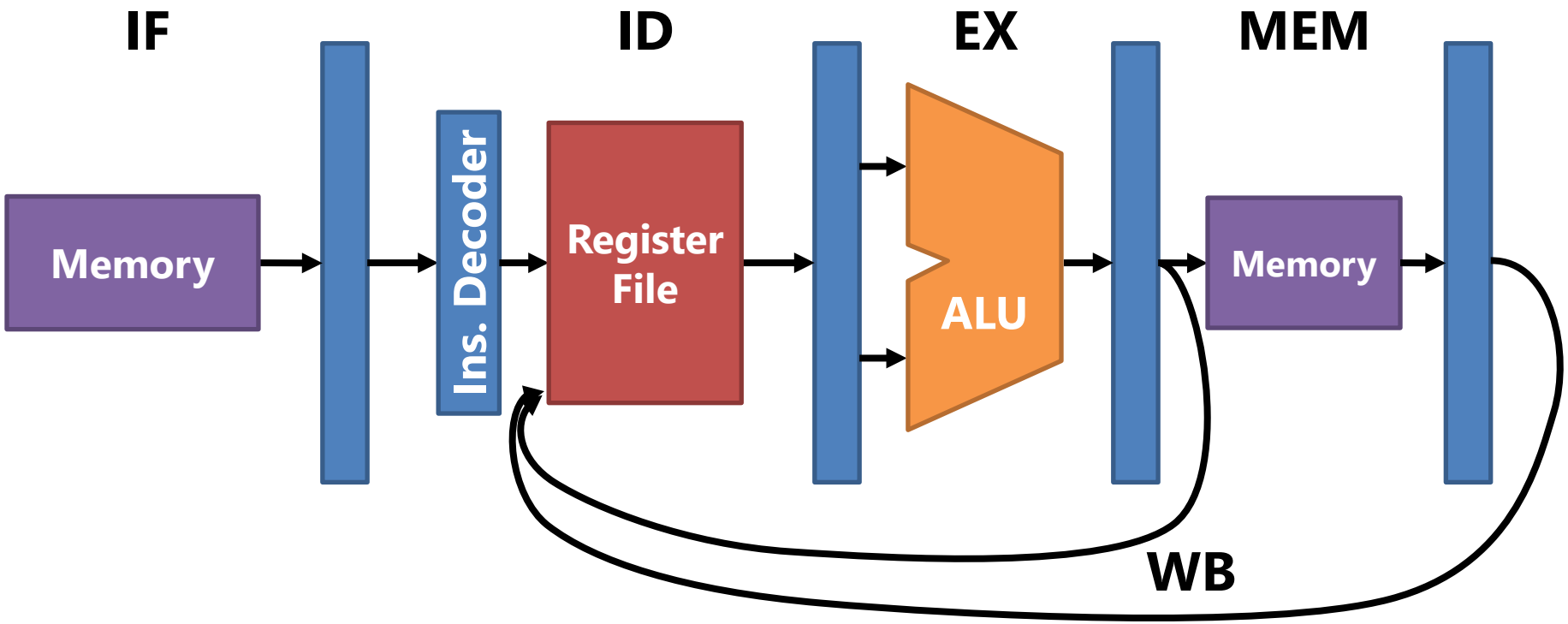
A Multi-cycle Implementation

- A Load instruction completes in 5 cycles.



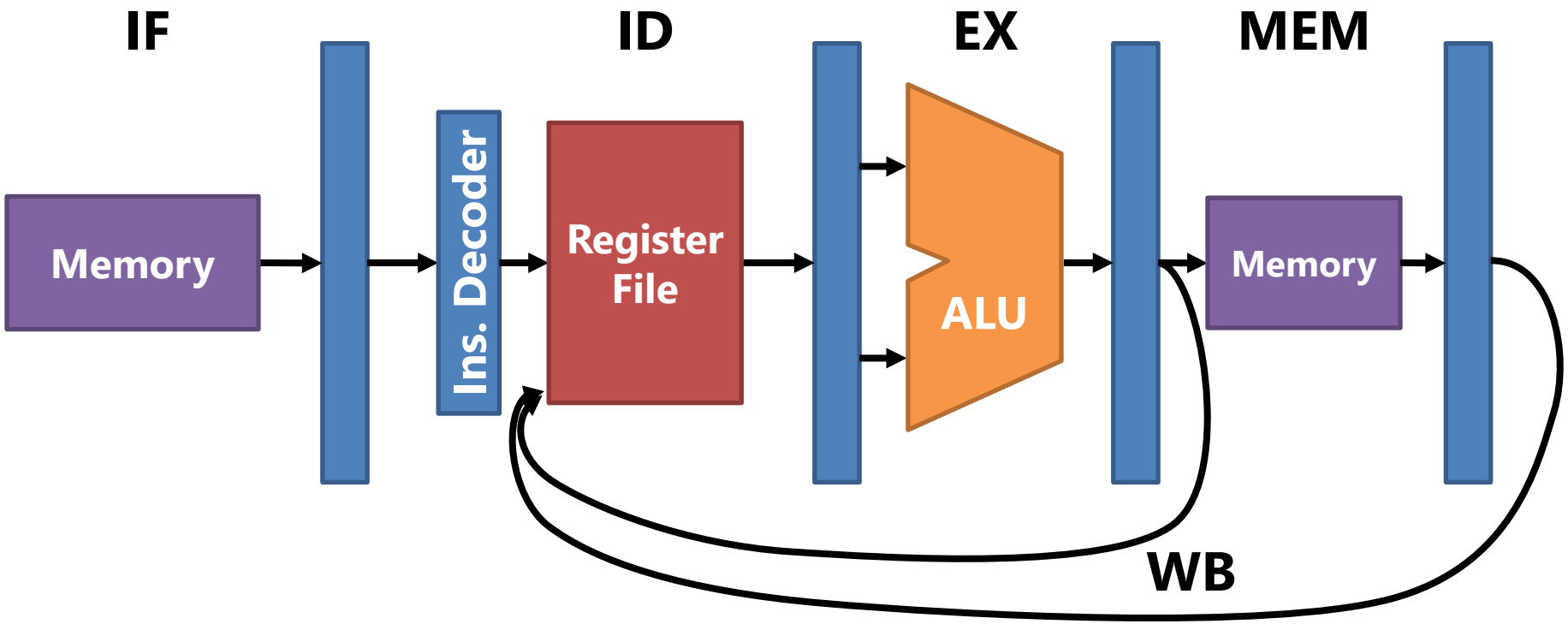
A Multi-cycle Implementation

- But a Store instruction completes in 4 cycles.



A Multi-cycle Implementation

- Register-to-register ALU instructions complete in 4 cycles too.



A Multi-cycle Implementation is Faster!

- Now each instruction takes different number of cycles to complete
  - **lw** takes 5 cycles: IF/ID/EX/MEM/WB
  - **add** takes 4 cycles: IF/ID/EX/WB

- If each phase takes *1 ns* as before:
  - **lw** takes *5 ns* and **add** takes *4 ns*

Q) If **lw** is 1% and **add** is 99% of instruction mix,
what is the average instruction execution time?

A) 0.01 * *5 ns* + 0.99 * *4 ns* = *4.01 ns* (25% faster than single cycle)

** Caveat: delay due to the added latches not shown, but net win*

And we can do even better!

- Did you notice?
  - When an instruction is on a particular phase (e.g. IF) …
  - … other phases (ID/EX/MEM/WB) are not doing any work!

- Our CPU is getting chronically ***underutilized***!
  - If CPU is a factory, 80% (4/5) of the workers are idling!

- Car factories create an assembly line to solve this problem
  - No need to wait until a car is finished before starting on next one
  - Our CPU is going to use a ***pipeline*** (similar concept)